

Part 2: Input Memory Module

AI Policy Reminder

Reminder: please see the project AI Policy in Appendix A of the Project Overview document, which explains how AI may and may not be used in this project, and how you should log and disclose AI usage.

The goal of Part 2 is to construct and test the Input Memory module. This module holds two memories, which will be used to buffer the input matrices. It also includes registers and control logic. This module is parameterized by $WIDTH$, R , C , and K . (See Project Overview Section 4.2 for the requirements on these parameters.) Figure 1 shows the top-level block diagram of this module.

Internally, this module will contain control logic and memories for holding the input matrix X and weight matrix W . It also contains registers to hold the values of S , B , and Q . On the left of Figure 1 you will see an AXI-Stream input interface, which will receive the input data. On the right of Figure 1 you will see X_* and W_* ports. These provide an interface that allows the rest of your convolution system to read the data stored in the input memories. Here is a specification of each port:

- $AXIS_TDATA$, $AXIS_TUSER$, $AXIS_TVALID$, and $AXIS_TREADY$ collectively form an AXI-Stream interface used to load input data into the module. See Project Overview Section 5 for a description of AXI-Stream, and see Section 1 below for information on how this AXI-Stream interface will be used to load X , W , B , Q , S , and new_W (the latter will be introduced below).
- The $inputs_loaded$ signal is an output that your module will use to indicate when its internal memories hold complete inputs (X and W matrices and values of S , B , and Q). We will refer to the time when $inputs_loaded=1$ as the *Inputs Loaded* phase. (In Part 3, your top-level convolution system will use this signal to determine when it is time to begin performing the computation.)
- X_read_addr and X_data are used to read data from the X input matrix memory. When your module is in the *Inputs Loaded* phase, then the X_read_addr signal will select an address in the X memory, and

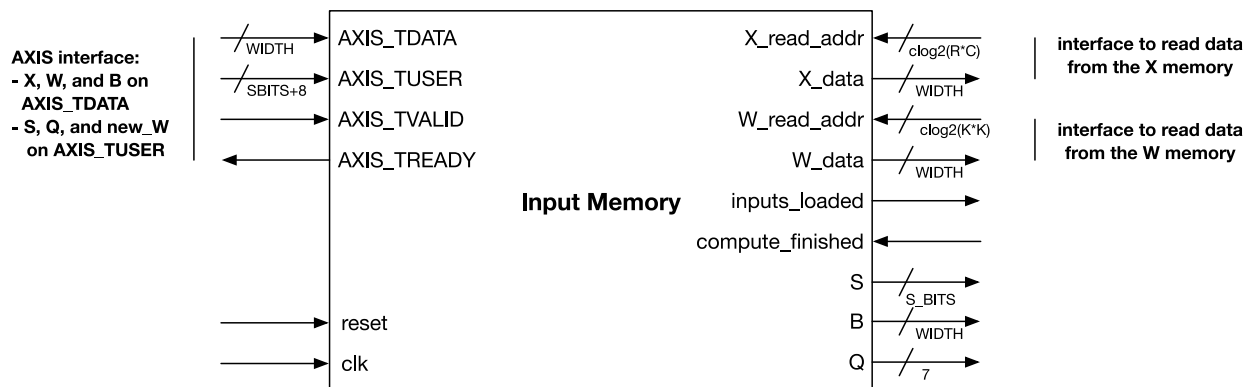


Figure 1: Input Memory module. $S_BITS = \lceil \log_2(MAXS+1) \rceil$, where $MAXS$ is defined in Section 4.1.

the X memory's output will be provided on X_data . (When your system is not in the *Inputs Loaded* phase, it can ignore these signals.)

- W_read_addr and W_data are used to read data from the W weight matrix memory in the same way as the $X_$ versions of these signals above.
- S is an output that indicates the stride used for the convolution. (Recall from Project Overview Section 3.3 that the stride S controls how far the weight matrix “moves” across X as you compute output values. S can be any value such that $1 \leq S \leq MAXS$, where $MAXS$ is derived from R , C , and K .) We require the S output to be correct and valid anytime $inputs_loaded$ is 1.
- B is an output that holds the bias value B . We require the B output to be correct and valid anytime $inputs_loaded$ is 1.
- Q is an output that holds the value of Q : the number of bits the MAC unit will use to scale its accumulated result down before it leaves your system (as defined in Part 1). Like B , Q is a value that arrives as an input and is held internally in a register. We require the Q output to be correct and valid anytime $inputs_loaded$ is 1.
- The $compute_finished$ signal is an input that is used to tell the Input Memory module when the system is done computing the convolution on the matrices stored in memory. When $compute_finished$ is asserted on a positive clock edge, your module should set $inputs_loaded$ to 0 and go back to its initial state to begin taking in new input values. The $compute_finished$ signal should only ever be 1 when $inputs_loaded$ is 1. (So your system can safely ignore $compute_finished$ when $inputs_loaded$ is 0.)

1. Input Protocol

Inputs will be provided to this module via the AXI-Stream interface signals shown on the left of Figure 1. Recall the discussion of AXI-Stream from Project Overview Section 5.

There is some complexity to the input protocol, so we will break this description down into three parts. Each part of the input protocol corresponds to one *stage* of the provided testbench (Section 6.2). This means that, rather than trying to build support for the entire protocol at once, you can implement and test a little bit at a time. (In other words, you can choose to start by designing and implementing just the functionality from Section 1.1 and test it fully before moving on to Section 1.2.) The following sections describe portions of the protocol:

- Section 1.1: loading the W and X matrices and the bias value B . (*Testbench Stage 1.*)
- Section 1.2: loading the values of S and Q . (*Testbench Stage 2.*)
- Section 1.3: reusing previously loaded W , B , Q , and S . (*Testbench Stage 3.*)

1.1. Loading the W Matrix, the Bias Value B , and the X Matrix

When loading the matrices, your system will use the $AXIS_TVALID$, $AXIS_TREADY$, and $AXIS_TDATA$ signals. If $AXIS_TVALID$ and $AXIS_TREADY$ are both equal to 1 on a positive clock edge, then your system has received data. Your system will control the value of $AXIS_TREADY$, so you must set it to 0 when the system is not ready for new inputs.

Your system will first load the values of the W matrix, followed by the scalar value B , followed by the X matrix. Matrix values will be provided in row-major order. This means that the first row will be provided,

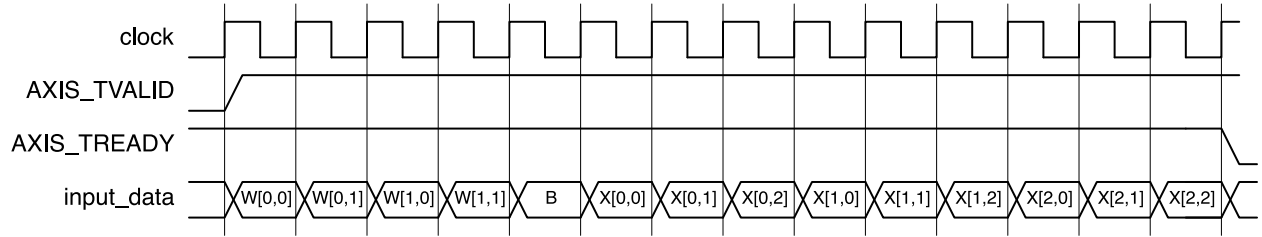


Figure 2: Example of loading inputs, assuming $R = 3$, $C = 3$, and $K = 2$. (To save space, array indices are shown as $W[a,b]$ instead of $W[a][b]$.)

then the second row, and so on. Your module must store the matrix values in the internal matrix memory in this order.

So, the complete sequence of input words for one convolution is:

$W[0][0], W[0][1], \dots, W[K-1][K-1], B, X[0][0], X[0][1], \dots, X[R-1][C-1]$

which is $K \cdot K + 1 + R \cdot C$ words in total.

This process is illustrated in Figure 2 for $R=3$, $C=3$, and $K=2$. This means that W is a 2×2 matrix and X is 3×3 . Note that in this example, $AXIS_TVALID$ and $AXIS_TREADY$ are both 1, so the timing is simple. However, if $AXIS_TVALID$ were to become 0, then nothing would be transmitted at that time, and the system must stall. Note also that after receiving the last value of X , this module will set $AXIS_TREADY$ to 0 (as seen on the right of the figure), since it has finished loading the inputs.

If you use the testbench (Section 6) with `STAGE` set to 1, it will test this reduced version of the input protocol (loading W , B , and X) and check the results.

1.2. Loading the Values of S and Q

Your system will load the values of S and Q alongside the first value of the W matrix by using the $AXIS_TUSER$ signal, along with the signals used above.

We will define $AXIS_TUSER$ as S_BITS+8 bits wide, where $S_BITS = \lceil \log_2(MAXS+1) \rceil$.¹ ($MAXS$ is a parameter defined in Section 4.1 below.) The bits of $AXIS_TUSER$ carry three values packed together:

- Its least significant bit, $AXIS_TUSER[0]$, will be saved for a different purpose that we will specify in Section 1.3 below.
- The next S_BITS bits, $AXIS_TUSER[S_BITS:1]$, will provide the value of stride S at the same time that you are loading the first entry of the W matrix ($W[0][0]$). In the rest of this specification, we will refer to these bits as $TUSER_S$.
- The top 7 bits, $AXIS_TUSER[S_BITS+7:S_BITS+1]$, will provide the value of Q , also at the same time as the first entry of W . We will refer to these bits as $TUSER_Q$.

¹ $\lceil \log_2(x) \rceil$ computes $\lceil \log_2 x \rceil$, where $\lceil \cdot \rceil$ denotes the “ceiling operator,” which rounds up to the next integer.

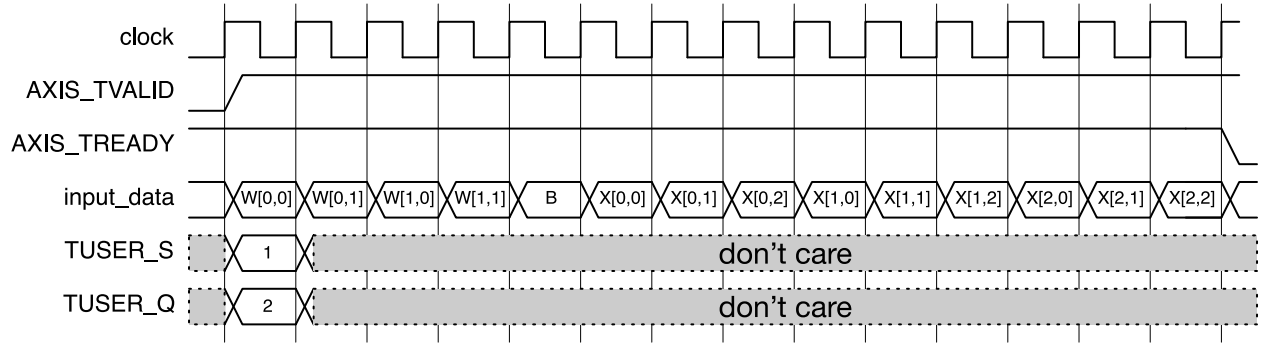


Figure 3: Example of loading inputs including loading of S , assuming $R = 3$, $C = 3$, and $K = 2$.

For simplicity, I suggest you make the following assignments in your SystemVerilog description of this module:

```

logic [S_BITS-1:0] TUSER_S;
assign              TUSER_S = AXIS_TUSER[S_BITS:1];

logic [6:0]        TUSER_Q;
assign              TUSER_Q = AXIS_TUSER[S_BITS+7:S_BITS+1];

```

Then, in your code you can simply refer to the **TUSER_S** and **TUSER_Q** signals.

These bits of **AXIS_TUSER** are only defined/valid during the first cycle of loading the W matrix. That is, **TUSER_S** holds the value of S and **TUSER_Q** holds the value of Q only during the time that your system is reading the first matrix entry. At all other times, they should be treated as don't-cares. The testbench will drive random values onto these signals at all other times, so if your design reads them when it shouldn't, the testbench will detect the error. This process is illustrated in Figure 3 for $R=3$, $C=3$, and $K=2$.

The testbench with **STAGE** set to 2 will test the input protocol including S and Q as described here.

1.3. Reusing the Previously Loaded W , B , Q , and S

Finally, your system must be able to reuse the previous weight matrix W , bias B , scaling value Q , and stride S , rather than loading new ones. This can allow operations where the system loads a weight matrix and its accompanying values and then uses them to perform convolution with many different X matrices without having to reload all inputs.

To make this possible, we need to define a control bit as part of the input stream to tell the system when it is loading a new set of these values, and when it should use the old ones already stored. For this we will use **AXIS_TUSER[0]**, the least significant bit of **AXIS_TUSER**. We will call this signal **new_W**.

For simplicity, you may want to make the following assignment in your SystemVerilog description of this module:

```

logic new_W;
assign new_W = AXIS_TUSER[0];

```

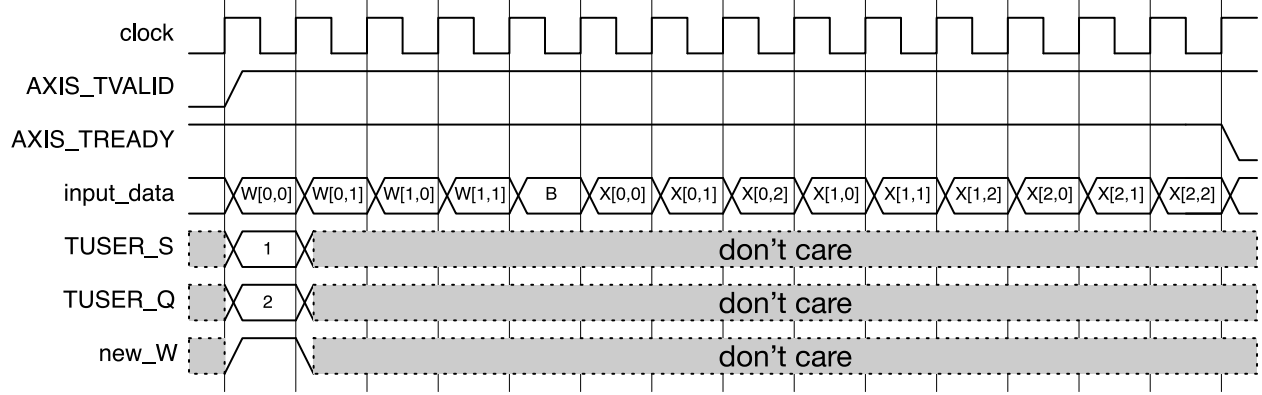


Figure 4: Example, with $R = 3$, $C = 3$, and $K = 2$. During the first valid cycle we read $new_W=1$ and $S = 1$, and $Q = 2$. Because $new_W=1$, we know that $AXIS_TDATA$ first represents the values of W , followed by B , followed by X .

The new_W signal only matters during the first cycle of input data transfer. That is, as your system begins processing inputs for a new convolution, it will check new_W on the first valid input cycle. If $new_W=1$ at that time, then the $AXIS_TDATA$ at that time represents the first value of the W matrix, $TUSER_S$ at that time represents the corresponding value of S , and $TUSER_Q$ represents the corresponding value of Q . If $new_W=0$ at that time, then the data represents the first value of the X matrix, and the system will continue to use the old W matrix and associated B , Q , and S already stored from the last operation.²

The new_W signal is only meaningful during the first cycle of data transfer; at all other times, its value is ignored. Figure 4 shows the process when $new_W=1$, so the first cycle carries the new S and Q values on $AXIS_TUSER$ while $AXIS_TDATA$ begins a new W matrix, followed by B , followed by matrix X . Below that, Figure 5 shows an example where $new_W=0$, so the system ignores $TUSER_S$ and $TUSER_Q$, skips reading W and B , and immediately reads X .

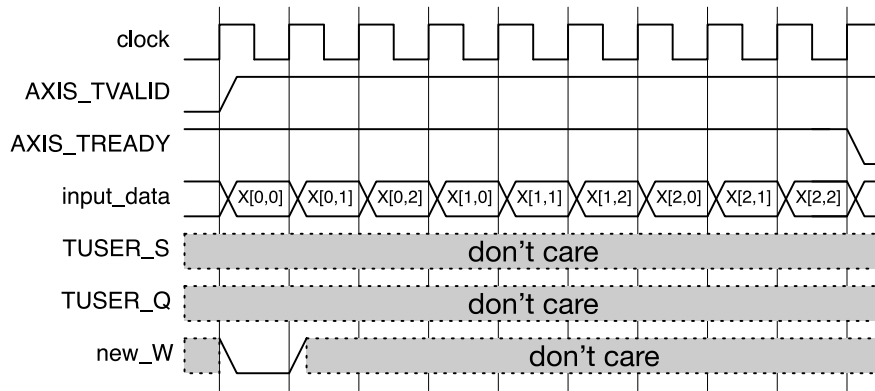


Figure 5: Example, with $R = 3$ and $C = 3$. Now during the first cycle we see that $new_W=0$, so we will reuse the old values of W , B , Q , and S that were previously stored. We see that $AXIS_TDATA$ immediately provides the values of X .

²This only makes sense if there is already a previously stored set of inputs. In other words, if this is the first operation after reset, there is nothing to reuse. We will treat this situation ($new_W=0$ on the first operation after reset) as undefined behavior. You do not need to worry about what your system does in this case, and the testbench will not try to do this.

Keep in mind that all transfers are done using the AXI-Stream protocol. This means that `AXIS_TDATA` and `AXIS_TUSER` are only transferred on positive clock edges when `AXIS_TVALID` and `AXIS_TREADY` are both equal to 1. Your system will set the value of `AXIS_TREADY`, so you must set it to 0 when the system is not “ready” for new inputs.

When the testbench is set to STAGE 3, it will test this entire Part 2 system, including the full input protocol.

2. Module Operation

Here we will give a brief overview of this module’s phases of operation and the sequence of steps it should take. You will construct control logic that will make the system perform the following process. Your control logic will likely need to include an FSM and counters. The following phases are not necessarily the only FSM states your system will use, but you should use them as a guide for how your control logic for this module should operate.

1. *Input W Matrix:* In this phase, the module will take in W matrix data via its AXI-Stream input interface. This data will be stored in the W memory. Your module will also store the values of S and Q during the first cycle of this phase. This phase is complete after the entire W matrix is loaded (K^2 values). This step will be skipped if `new_W` is 0; in that case, you would jump to the Input X Matrix phase. (See Section 1.3 above.)
2. *Input B:* In this phase, the module will take in the B (bias) value via the AXI-Stream input interface. Remember, B is a single (scalar) value, so only one word is received. This word will be stored in the B register. This phase is complete after this input is loaded. This step is skipped if `new_W` was 0 at the start of the input operation above.
3. *Input X Matrix:* In this phase, the module will take in X matrix data via its AXI-Stream input interface. This data will be stored in the X memory. This phase is complete after the entire X matrix is loaded ($R \times C$ values).
4. *Inputs Loaded:* This phase begins after the inputs are complete. Your system will output `inputs_loaded=1` during this phase. In this phase, your system should ensure that the S , B , and Q output signals hold the correct values. During this phase, your system is not ready to receive new inputs, so it must output `AXIS_TREADY=0`.

During this phase, your system will allow the memory read interfaces (on the right side of Figure 1) to read data from the matrix memories. This means that during this phase, the `X_read_addr` and `W_read_addr` input signals should be used to provide addresses to their respective memories.

While in this phase, your system should monitor the `compute_finished` input signal. When `compute_finished==1` on a positive clock edge, your system should exit this phase, set `inputs_loaded` to 0, and go back to the beginning, waiting for new input data.

3. Memory

Your Part 2 design will contain two memory instances: one to hold X and one to hold W . We will provide you with a memory module to use. Section 3.1 shows you that memory and describes its function. Then Section 3.2 describes how you will use these memory modules in your Part 2 design.

3.1. Memory Module Code and Operation

Your memories will use a single address port, and they read and write synchronously. Use the following module for both memories. You can also find a file with this code in it at:

```
/home/home4/pmilder/ese507/proj/part2/memory.sv
```

You may not modify this memory module (although you can feel free to copy/paste the code into another .sv file if that's more convenient).

```
module memory #(
    parameter WIDTH=16, SIZE=64,
    localparam LOGSIZE=$clog2(SIZE)
) (
    input [WIDTH-1:0]      data_in,
    output logic [WIDTH-1:0] data_out,
    input [LOGSIZE-1:0]    addr,
    input                  clk, wr_en
);

    logic [SIZE-1:0][WIDTH-1:0] mem;

    always_ff @(posedge clk) begin
        data_out <= mem[addr];
        if (wr_en)
            mem[addr] <= data_in;
    end
endmodule
```

There are several important things to understand about this memory:

- The memory has one read port, one write port, and one address input used for both reads and writes. (So, you cannot read and write to different locations of this memory at the same time.) All reads and writes are synchronous (that is, they will occur on a positive clock edge).
- The memory's parameters are WIDTH (the number of bits per word) and SIZE (the number of words).

Figure 6a demonstrates the timing of reading from the memory. On each positive clock edge, the system samples the value on `addr`. After the clock edge, the memory will output the value in memory at that location. In this diagram, `mem[7]` represents the value stored in address 7 of the memory.

Figure 6b demonstrates the timing of writing to memory. In this example, you are first writing the value 3 to address 6. Then, on the following cycle, no write is performed because the `wr_en` signal is 0 on the clock edge. Then, value 4 is written to address 1 on the third positive clock edge.

Recall from class that this RTL memory module will not synthesize to SRAM. Instead, it will result in a memory structure built out of flip-flops. If these memories were large, this would be very inefficient.

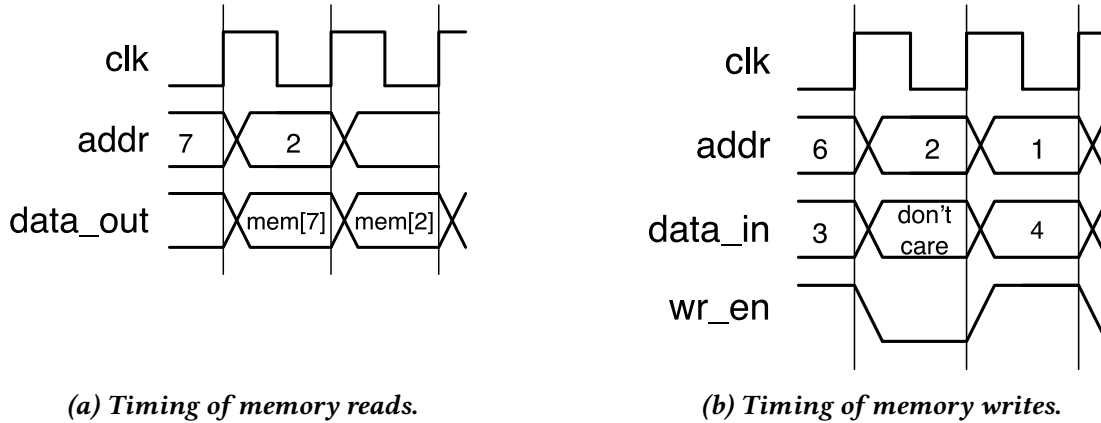


Figure 6: Timing of memory reads and writes.

However, the memories you will need in this project will be fairly small, so we will simply let the logic synthesis tool implement them using registers.

3.2. Using Memories in Your Part 2 Design

Your design will use two memories: one to hold X and one to hold W . Both should be instantiated with their `WIDTH` parameter set to your system's `WIDTH` parameter. The X matrix memory should have `SIZE=R*C`, and the W matrix memory should have `SIZE=K*K`. Your system should store the matrices in row-major order (see Section 1.1).

- The `data_in` port of both memories should connect directly to the `AXIS_TDATA` input of this module.
- The `data_out` port of the X memory should connect directly to the `X_data` output of this module, and the W memory's `data_out` should connect to `W_data`.
- The `addr` and `wr_en` ports of these memories should be driven by your module's internal logic.

Once your module is in the *Inputs Loaded* phase (and it is setting the `inputs_loaded` output signal to 1), your control logic should allow the memory read interfaces (on the right side of Figure 1) to read data from the memories. That is, when `inputs_loaded==1`, then the X matrix memory's address input should equal `X_read_addr`, and the W memory's address input should equal `W_read_addr`. At all other times (when `inputs_loaded` is 0), your internal logic will determine the addresses on these memories.

4. Details about S , Q , and B

As described above, your Part 2 design will use registers to hold S , Q , and B . This section discusses some considerations related to these signals.

4.1. Stride S

The value of S (the stride of the convolution) is an input to your system. As described in Section 1.2 above, your system will use bits of the `AXIS_TUSER` signal to read the value of S . Your `input_mems` module should contain a register to hold this value, and this register's output should directly go to the S output of this module. Your system must guarantee that the S output is correct any time `inputs_loaded==1`.

One thing we haven't addressed yet is the width of S . We can determine this by first computing the

maximum meaningful value that S can have. We will call this MAXS .³

$$\text{MAXS} = \max(R, C) - K + 1$$

Given this, we can now determine the number of bits needed to allow S to reach MAXS , which we will define as $\text{S_BITS} = \lceil \log_2(\text{MAXS}+1) \rceil$.

Since both MAXS and S_BITS can be computed based only on parameters R , C , and K , we can define them as local parameters in your module. You can use this in your code (in the parameter lists where you define parameters and `localparams`):

```
localparam MAXS    = ((R > C) ? R : C) - K + 1,  
localparam S_BITS  = $clog2(MAXS+1),
```

Your system must work for any value of S in the range $1 \leq S \leq \text{MAXS}$, and the testbench will only provide values in that range.

4.2. MAC Scaling Value Q

As described in Section 1.2 above, your module will use bits of the `AXIS_TUSER` signal to read the value of Q (which is used by your MAC module as described in Part 1). Your module should contain a register to hold the value of Q , and this register's output should go to the Q output of this module. Your system must guarantee that the Q output is correct any time `inputs_loaded==1`. The Q signal is always 7 bits wide.

4.3. Bias B

The bias B will be stored in a `WIDTH`-bit register, which should directly drive the module's output signal named B . The register should be updated when the B value is provided (as described in Section 1.1 above). Your system must guarantee that the B output is correct any time `inputs_loaded==1`.

³To understand this definition, notice that doing a convolution with stride MAXS would result in a 1×1 output. (To see why, plug this definition of MAXS into the R_{out} and C_{out} expressions from the Project Overview document.) For this reason, increasing the stride beyond MAXS does not change the result.

5. Code

Store all of your files for Part 2 in a subdirectory called `part2/`. Implement this design in a file called `input_mems.sv`. Your system should use the memory module given above. Use the following top-level module name, port names, and port declarations:

```
module input_mems #(
    parameter WIDTH = 24,
    parameter R = 9,
    parameter C = 8,
    parameter K = 4,
    localparam MAXS = ((R > C) ? R : C) - K + 1,
    localparam S_BITS = $clog2(MAXS+1),
    localparam X_ADDR_BITS = $clog2(R*C),
    localparam W_ADDR_BITS = $clog2(K*K)
)(
    input clk, reset,
    input [WIDTH-1:0] AXIS_TDATA,
    input AXIS_TVALID,
    input [S_BITS+7:0] AXIS_TUSER,
    output logic AXIS_TREADY,
    output logic inputs_loaded,
    input compute_finished,
    output logic [S_BITS-1:0] S,
    output logic signed [WIDTH-1:0] B,
    output logic [6:0] Q,
    input [X_ADDR_BITS-1:0] X_read_addr,
    output logic signed [WIDTH-1:0] X_data,
    input [W_ADDR_BITS-1:0] W_read_addr,
    output logic signed [WIDTH-1:0] W_data
);
```

6. Testbench

You are provided with a testbench to test your `input_mems` module. This testbench will generate random input data and then feed the random inputs to your module using the AXI-Stream interface (with random timing on the `AXIS_TVALID` signal). The testbench will wait until your module asserts the `inputs_loaded` signal, and then it will read the data back from your module's internal memories, checking that the data retrieved matches what was transmitted. You can find the testbench in the following file:

```
/home/home4/pmilder/ese507/proj/part2/input_mems_tb.sv
```

Copy this file and the `simParams2` and `genParams2` files into your `part2/` work directory. Before you use it, please read the following brief explanation of how the testbench works. Then read through the testbench files and their comments.

6.1. Testbench Structure and Scripts

The testbench module includes several parameters:

- `WIDTH`: the number of bits for each data word
- `R`: the number of rows in the X matrix
- `C`: the number of columns in the X matrix
- `K`: the number of rows/columns in the W matrix
- `TESTS`: the number of input sets to simulate
- `INPUT_TVALID_PROB`: a decimal value that represents the probability that the testbench will assert `AXIS_TVALID` on any given cycle. The probability should be between 0.001 and 1, inclusive, or select 0 to randomize it.

For example, if you set `INPUT_TVALID_PROB` to 0.3, then there will be a 30% chance that the testbench will assert `AXIS_TVALID` (and try to transmit input data) on each cycle. If you set this parameter to 0, the testbench will randomly pick a probability at the beginning of the simulation (and tell you what it chose).

I recommend that you first get your system working with `INPUT_TVALID_PROB=1` before trying lower values. Don't skip doing simulations with low probability, though. (You will need these to catch bugs in your design that only happen during stalling.)

- `STAGE`: an integer from 1 to 3 that chooses how much of the input protocol specification to test. This allows you to implement your module in pieces and test progressively as you build, instead of testing the whole thing at once. Details are in Section 6.2 below.

The testbench uses a SystemVerilog class called `testdata`. Here, the class holds the input data for an entire convolution operation. That is, it holds the values of X , W , B , Q , S , and `new_W`. The testbench uses SystemVerilog's internal randomization capabilities to randomize the values of these inputs.

For each test, the testbench will randomize the test data and feed it into the system via the AXI-Stream interface. The timing of this transaction will be randomized based on `INPUT_TVALID_PROB`.

After feeding in a set of test data, the testbench will wait for the DUT to set `inputs_loaded` to 1. Then, the testbench will use the DUT's `X_read_addr` and `W_read_addr` inputs to read the stored data back from the DUT's internal memories, checking their values, as well as S , B , and Q .

Note that the testbench drives *random* values onto the bits of `AXIS_TUSER` during any time when those bits are "don't cares" (see Figure 4 and Figure 5).

The testbench will count the number of errors. It is set to stop running if the error count reaches 100. (You can easily modify this if you want to.) Additionally, the testbench will track the progress of the simulation; if progress stalls (for example, if a bug in your logic prevents the test from continuing) for one million cycles, the testbench will halt and print a message.

Run the testbench with the `simParams2` script, just as you used `simParams1` in Part 1. (Review the Part 1 description for more information, including what `[GUI]` means and how to reproduce a run from its random seed.)

```
./simParams2 [WIDTH] [R] [C] [K] [INPUT_TVALID_PROB] [STAGE] [GUI]
```

where you will replace each bracketed item with the desired value of that parameter. Don't forget that if the `INPUT_TVALID_PROB` value is set to 0, then the simulation will randomize its value.

As in Part 1, `genParams2` only generates the `params.sv` file. You need it only if you are running the simulator by hand (for example, to reproduce a particular random seed):

```
./genParams2 [WIDTH] [R] [C] [K] [INPUT_TVALID_PROB] [STAGE]
```

(If you do compile by hand, note that there is no C file to compile here, since this testbench does not use DPI like Part 1 did.)

6.2. Building Your Design in Stages

To make it easier to debug your system, the testbench we provide includes a `STAGE` parameter that lets you implement and verify your logic in progressively larger stages. The stages are *cumulative*: the design you build for one stage is a portion of what you need for the next one. (Nothing you write at an earlier stage is thrown away.) Stages 1, 2, and 3 map to Section 1.1, Section 1.2, and Section 1.3, respectively.

- **Stage 1** tests the portion of the protocol in Section 1.1. It sends the full W , B , X sequence for each test, and it checks the contents of the X and W memories and the B register. It still doesn't check S or Q or ever test `new_W==0`.
- **Stage 2** tests the portion of the protocol in Section 1.2. In addition to the inputs used in Stage 1, it also adds S and Q on the `AXIS_TUSER` input and checks their correctness. It still doesn't test `new_W==0`.
- **Stage 3** tests the full input protocol (Section 1.3). In addition to what Stage 2 did, it now includes inputs where `new_W==0`, and it checks everything.

Stage 3 tests the full system; this is what your work will be evaluated on. Although I recommend it, you are not required to build and test your system one stage at a time. If you prefer to implement the entire thing and test it with Stage 3, you can choose to do so.

Use the module port list given in Section 5 for all stages, even though some of the outputs are unused in the earlier stages. (For example, in Stage 1, your system won't need to do anything with the S and Q outputs, so you could just leave those unassigned until you are ready to work on Stage 2.)

7. Part 2 Submission

Simulate your design for a variety of different parameters. Importantly, make sure you do some tests with extreme values of `INPUT_TVALID_PROB`; you want to make sure that the system works correctly even if it has to stall frequently, and that it works correctly if data arrives as quickly as possible.

Synthesize your full design twice, with the following parameters:

- `WIDTH=24`, `R=9`, `C=8`, `K=4`

- WIDTH=10, R=15, C=13, K=7

For each design, correct any synthesis problems you find, and find the maximum possible clock frequency.

Don't forget that you need to set the parameter values in your top-level module. The `params.sv` file used in simulation only affects the testbench; to set the values in synthesis, you must make sure the default parameter values in your top-level module are set correctly. Save your synthesis reports with descriptive names and include them in your Part 2 submission.

Your Part 2 submission should include your code, your synthesis reports, your AI usage log, and a brief plaintext status report. Please see Section 6 in the Project Overview document, which explains the submission requirements and procedure. Additionally, in your status report, explain which testbench stages you used and where you had problems. (For example, maybe you tested with Stage 1, found and fixed some bugs, then Stage 2 worked immediately, and then Stage 3 had more bugs.)

8. Final Report

Your Final Report is due at the end of the semester, although I recommend you work on it as you complete each part of the project. In your Final Report, include a clearly labeled section for Part 2, where you answer the following questions. More information about the Final Report can be found in Section 6 in the Project Overview document.

1. This part of the project required you to design a significant amount of control logic that interacts with the AXI-Stream interface, the memories, the S , B , and Q registers, and the `inputs_loaded` and `compute_finished` signals. Carefully and thoroughly explain how this module works, especially your control logic. Your documentation should allow the reader to fully understand how your `input_mems` module works (and any submodules) without looking at the code.
2. The number of cycles required by this module is largely determined by:
 - the parameters (R, C, K)
 - whether this input set has `new_W==1` or `new_W==0`
 - how the testbench asserts `AXIS_TVALID`

However, there are places where you as the designer could make choices that affect the number of cycles required by your module. For example, if your system unnecessarily sets `AXIS_TREADY` to 0, or it adds extra cycles of delay between steps, the system will be less efficient.

One way to quantify this is to measure how long your system takes to complete a task. Run a simulation where you set `WIDTH=10`, `R=15`, `C=13`, `K=7`, and `INPUT_TVALID_PROB=1`. When `new_W==1`, the testbench will begin by feeding in the $K^2 = 49$ values of W , then the value of B , then the $R \times C = 15 \times 13 = 195$ values of X . In other tests where `new_W==0`, the system will simply feed in the $R \times C = 195$ values of X .

Simulate this design in QuestaSim's waveform view and count the number of cycles between when your design sets `AXIS_TREADY` to 1 and when it sets `inputs_loaded` to 1. Do this for a few sets of inputs where `new_W==1` and a few sets of inputs where `new_W==0`, and record the number of cycles in your report.

Hint: you can view the amount of simulated time that passes in the waveform and then divide it by 10ns to get the number of simulated clock cycles. In your report, give this cycle count for both `new_W` of 0 and 1. Or, you could write a testbench (or modify the existing testbench) to measure this for you.

In the report, quantify how efficient your system is with respect to the number of clock cycles by computing the following ratios:

- When `new_W==1`, use $\text{efficiency} = \frac{K \cdot K + 1 + R \cdot C}{\text{cycles}}$
- When `new_W==0`, use $\text{efficiency} = \frac{R \cdot C}{\text{cycles}}$

In these metrics, 1.0 is perfectly efficient; a good implementation will be close to this. Report both metrics and the data you collected.

If your efficiency number is not close to 1, where could your logic be improved?

3. In the previous question, you measured the cycle count. Now, use your understanding of your system's behavior to write *equations* for the cycle count with respect to R, C, and K. You should have one equation for `new_W==1`, and one equation for `new_W==0`. (If your efficiency is a perfect 1.0, then this step will be trivial.)
4. For the Part 2 submission (above), you synthesized the design with parameters:
 - WIDTH=24, R=9, C=8, K=4
 - WIDTH=10, R=15, C=13, K=7

For each set of parameters, report the clock frequency, area, power, and critical path location. Carefully explain each design's critical path. Don't just list its start and end points; explain what logic is included in the path and what it means. Does the critical path location change between these two designs? Explain why or why not.

5. If you worked with a partner, please carefully describe each partner's contribution to this part of the project. (If you did not work with a partner, skip this.)